

Strategies to Avoid Poor Local Minima in Nonlinear Optimization

Nicolas Heidbrink, IOR,
June 16, 2026



1. Problem Formulation

- Iterated Local Search

2. The Tunneling Algorithm

- How It Works
- Problems with the Algorithm
- Adaptations

3. Simulated Annealing

- How It Works
- Application in the Tunneling Algorithm



Problem Formulation

Section
1

Problem Formulation

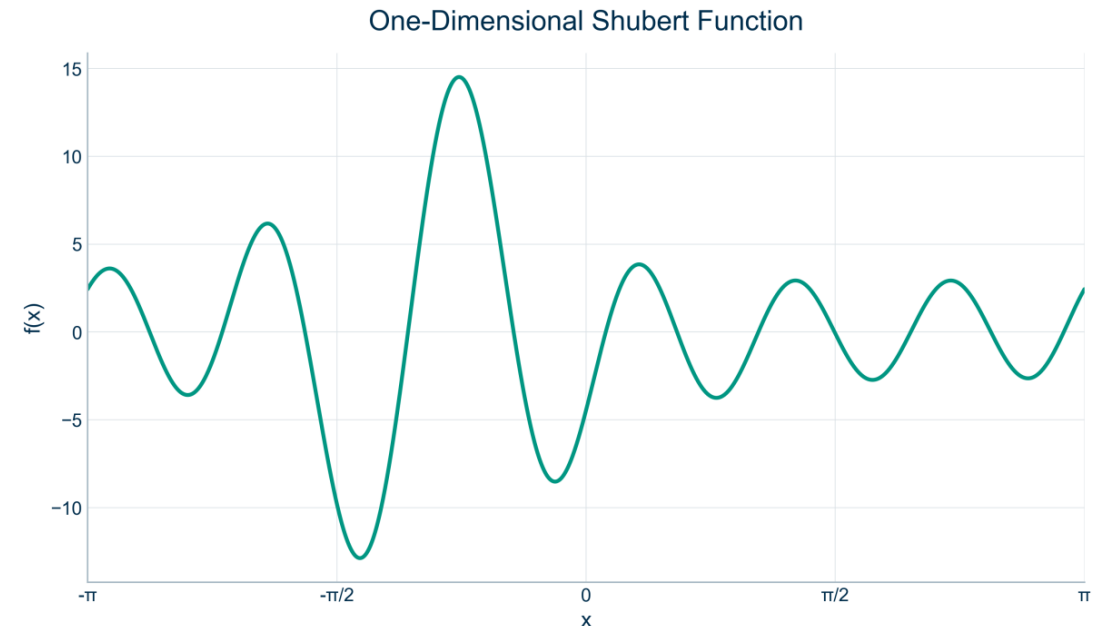
- Classic minimization problem subject to box constraints
- Typically multidimensional feasible sets
 - Our focus: $2 \leq n \leq 10$
- Objective functions often multimodal
- Gradient given
- Hessian matrix not given

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & x_i \in [l_i, u_i], \quad \forall 1 \leq i \leq n \end{aligned}$$

One-Dimensional Shubert Function

- Multimodal periodic function
- Global minimal point:
 - $f(-1.43 + 2k\pi) = -12.87, \quad k \in \mathbb{Z}$
- Global maximal point:
 - $f(-0.8 + 2k\pi) = 14.51, \quad k \in \mathbb{Z}$
- Three of each over $x \in [-10, 10]$

$$f(x) = \sum_{i=1}^5 i \cos((i+1)x + i)$$

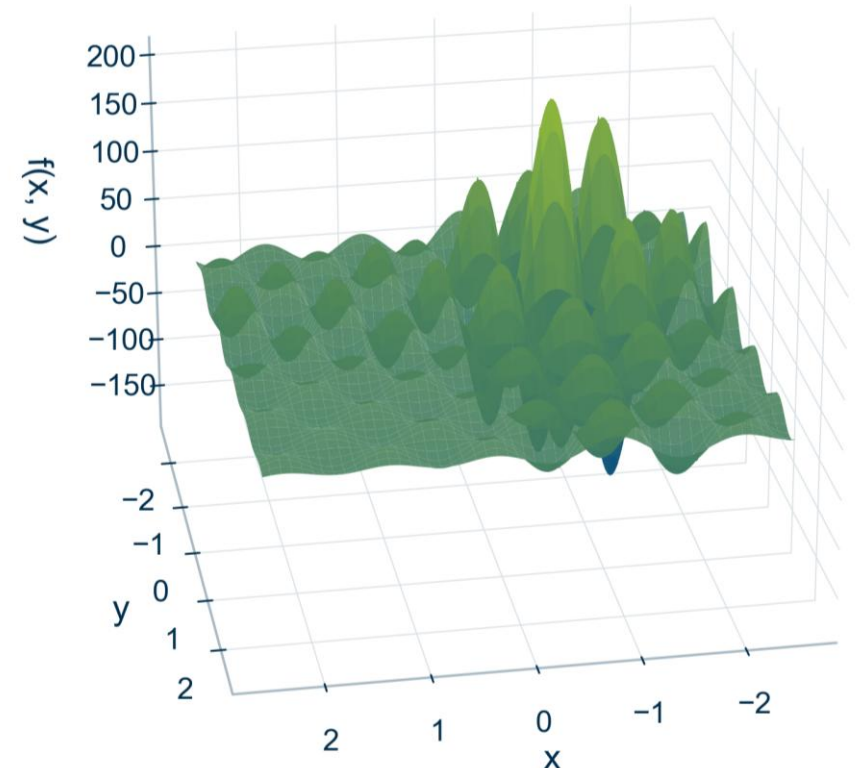


Multidimensional Shubert Function

- Multimodal periodic function
- Test function for global minimization algorithms
- Typically examined over $x \in [-10,10]^n$
- Formula for count of global minimal points:
 - $n \times 3^n$

$$f(\mathbf{x}) = \prod_{j=1}^n \left(\sum_{i=1}^5 i \cos((i+1)x_j + i) \right)$$

Two-Dimensional Shubert Function



Iterated Local Search

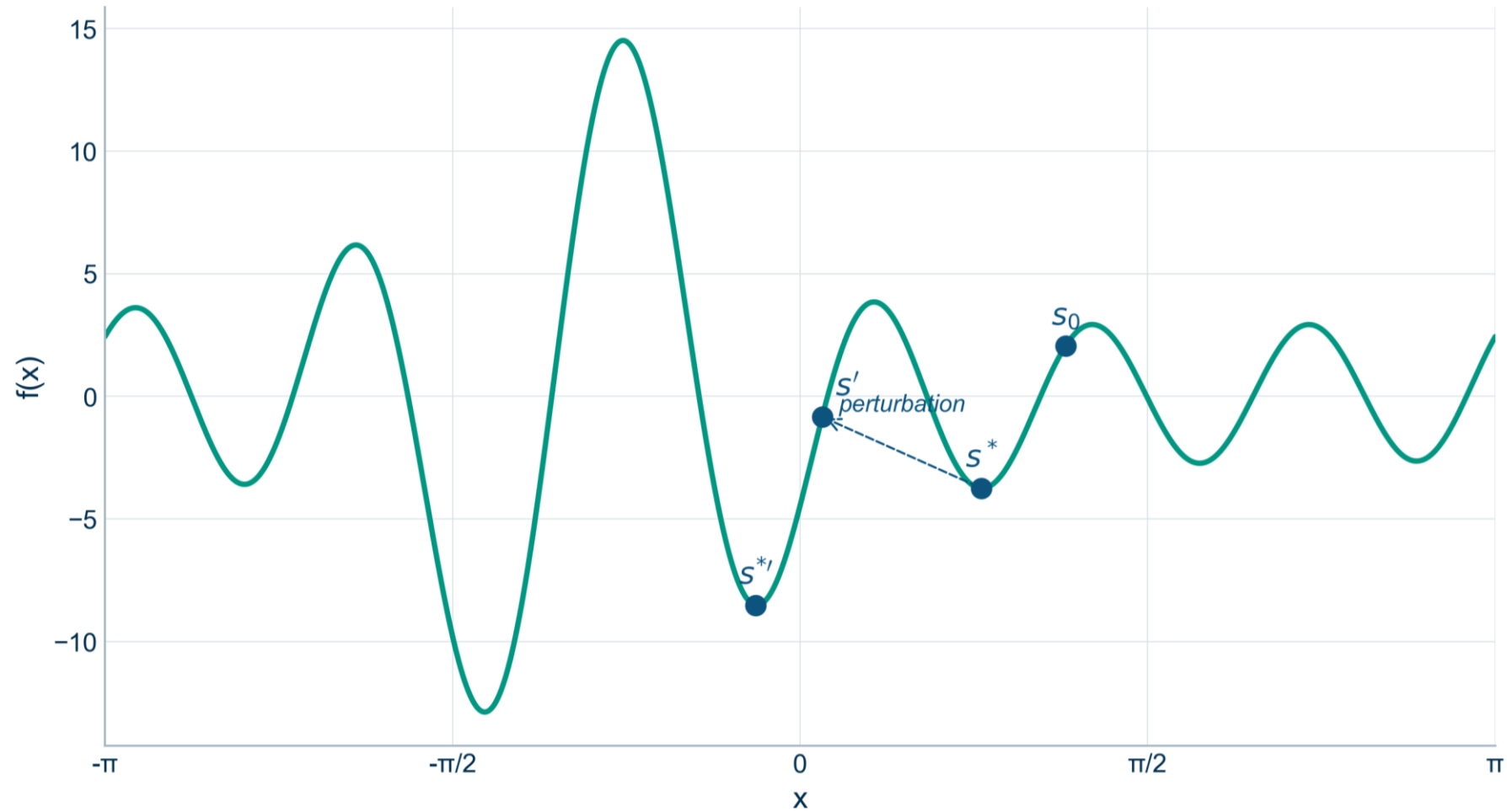
- Metaheuristic for global optimization
- Starting from a local minimal point s^* , there are three steps:
 - Find any other point s'
 - Search for a local minimal point from s' and save it as $s^{*'}$
 - Choose whether to continue from $s^{*'}$ or go back to s^*
- The sequence of s^* can be non-ascending if desired

Algorithm Iterated local search

```
1:  $s_0 = \text{GenerateInitialSolution}$ 
2:  $s^* = \text{LocalSearch}(s_0)$ 
3: repeat
4:    $s' = \text{Perturbation}(s^*, \text{history})$ 
5:    $s^{*' } = \text{LocalSearch}(s')$ 
6:    $s^* = \text{AcceptanceCriterion}(s^*, s^{*' }, \text{history})$ 
7: until termination condition met
```

Iterated Local Search

Example



Random Search Approaches

Multiple Random Start

- Random points in the feasible set are generated
- Local minimization algorithms are applied
- The resulting local minimal points with the lowest objective function value are presumed to be optimal
- Computationally expensive

Modified Multiple Random Start

- Random points in the feasible set are generated
- One by one, local minimization algorithms are applied
- If the objective function value of a random point is not as low as the optimal value discovered so far, it is skipped
 - It quickly results that most random points are skipped
- Unsuitable for functions with multiple global minimal points

Both methods are simple to implement but show worse performance than the tunneling algorithm in tests.



The Tunneling Algorithm

Section
2

Overview

Method for finding global minimal points of multidimensional multimodal functions

Invented in 1985 by A. V. Levy and A. Montalvo

Alters between two phases:

1. Minimization phase

- **Goal:** Find a local minimal point
- **Method:** Apply a descent method on the objective function

2. Tunneling Phase

- **Goal:** Find a new point at least as optimal as the previously found minimum
- **Method:** Apply a descent method on the tunneling function

Tunneling Function

$$T(x) = \frac{f(x) - f^*}{\{\prod_{i=1}^l [(x - x_i^*)^T (x - x_i^*)]^{\eta_i}\} [(x - x_m)^T (x - x_m)]^{\lambda_0}}$$

Meaning of terms:

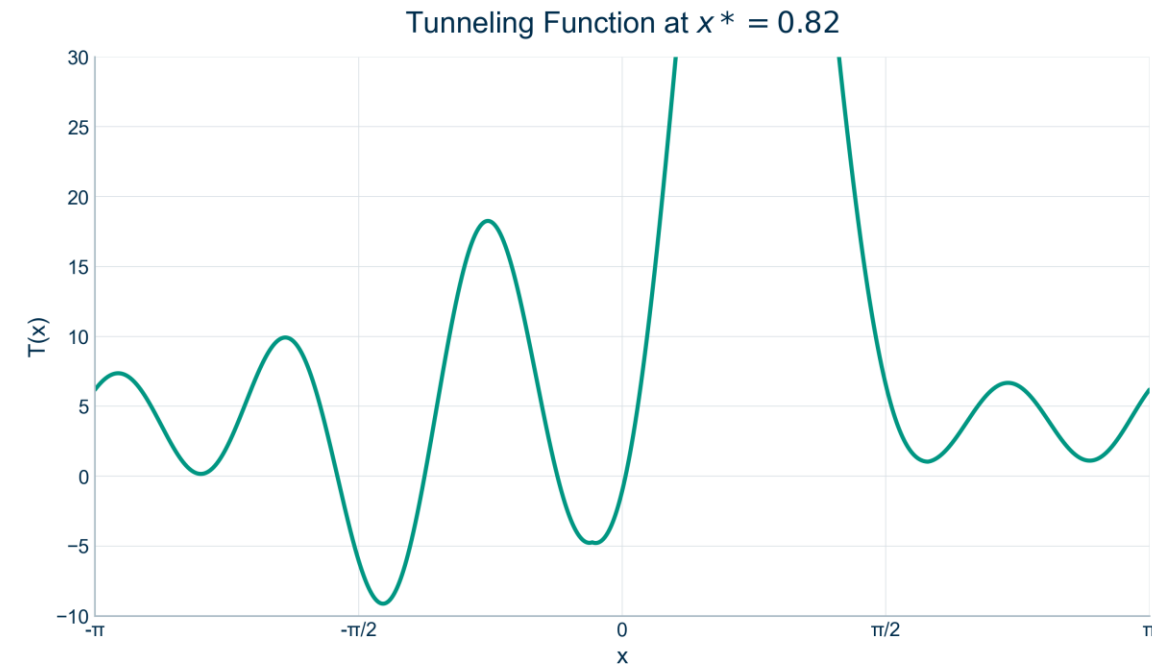
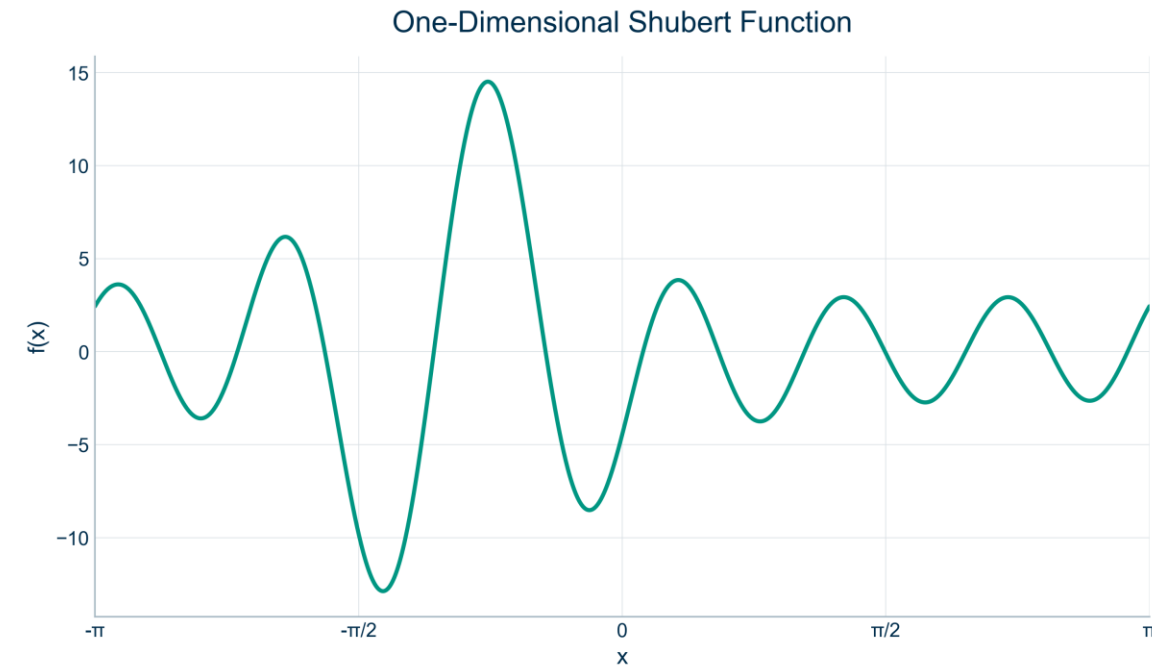
- x_i^* : List of known minimal points
- $f^* := f(x_i^*)$
- η_i : Exponent to create a pole around minimal points
- λ_0 : Exponent to create a movable pole

Sign of the tunneling function:

- $T(x) > 0$ if $f(x) > f^*$ (x less optimal than x_i^*)
- $T(x) = 0$ if $f(x) = f^*$ (x as optimal as x_i^*)
- $T(x) < 0$ if $f(x) < f^*$ (x more optimal than x_i^*)

Tunneling Function Example

- Pole at minimal point $x^* = 0.82$
 - Lower minimal points not known in this iteration
- No movable pole



First Term in the Denominator

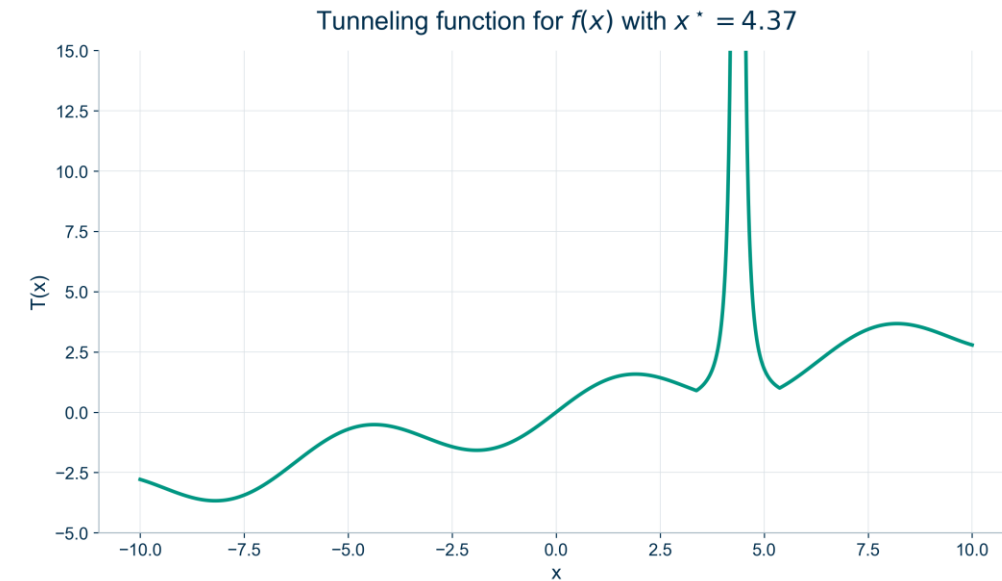
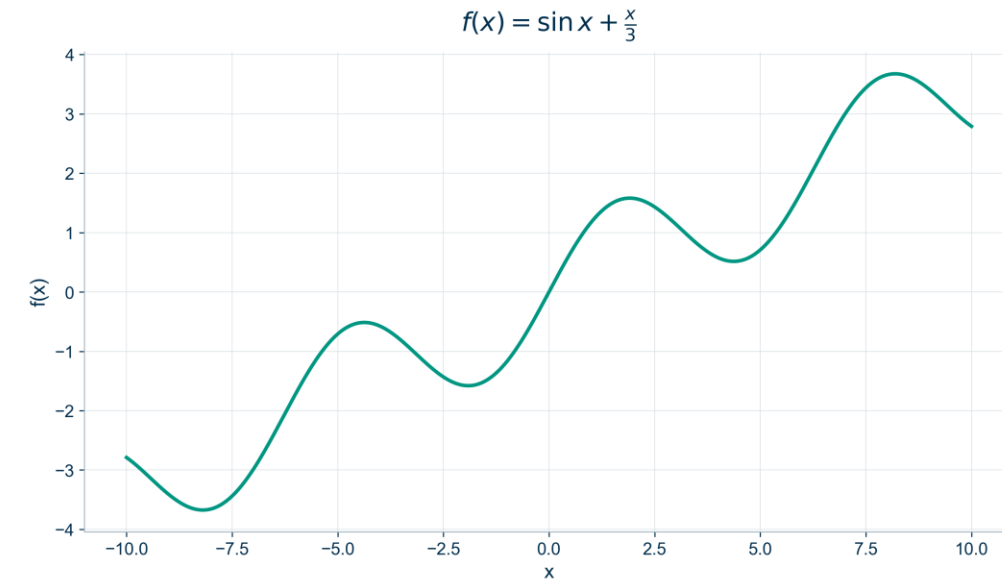
$$T(x) = \frac{f(x) - f^*}{\{\prod_{i=1}^l [(x - x_i^*)^T (x - x_i^*)]^{\eta_i}\} [(x - x_m)^T (x - x_m)]^{\lambda_0}}$$

- Adds poles at all x_i^*
- When a new minimum is found:
 - If it has a unique objective function value, it replaces all previous x_i^*
 - If it has the same objective function value, it is added to the list
 - An exponent λ_i is found to create a pole at x_i^*
 - η_i ensures the pole only influences points within a radius of 1 of x_i^* :

$$\eta_i = \begin{cases} 0 & \text{if } \|x - x_i^*\| \geq 1 + \varepsilon, \\ \lambda_i & \text{if } \|x - x_i^*\| \leq 1 - \varepsilon, \\ (\lambda_i/2)[1 + (1 - \|x - x_i^*\|)/\varepsilon] & \text{if } 1 - \varepsilon \leq \|x - x_i^*\| \leq 1 + \varepsilon \end{cases}$$

First Term in the Denominator

- Pole added around $x^* = 4.37$
- No point with a lower objective function value is known



How It Is Determined a Pole Was Added

$$\Delta x^T \nabla T(x) < 0$$

$$\Delta x^T \epsilon > 0$$

- Two above terms are evaluated
 - If not satisfied: λ_i increased
- If gradient descent is used, first equation trivially true
 - $\Delta x = -t \nabla T(x)$
- Δx : Displacement $x^{(1)} - x^{(0)}$ for a given λ_i
- $\nabla T(x)$: Gradient of tunneling function for a given λ_i
- ϵ : Small random perturbation from pole

Second Term in the Denominator

$$T(x) = \frac{f(x) - f^*}{\{\prod_{i=1}^l [(x - x_i^*)^T (x - x_i^*)]^{\eta_i}\} [(x - x_m)^T (x - x_m)]^{\lambda_0}}$$

- x_m placed between x_i and x_{i-1}
- λ_0 calculated to create a pole at x_m
- Aims to prevent stagnation around local minimal points of tunneling function
 - x_m implemented when necessary to negate minimal points

$$x_m^{(i)} := \begin{cases} x^{(i-1)} & \text{if } \|x^{(i)} - x^{(i-1)}\| < 1, \\ \xi_i x^{(i-1)} + (1 - \xi_i) x^{(i)} & \text{if } \|x^{(i)} - x^{(i-1)}\| \geq 1 \end{cases}$$

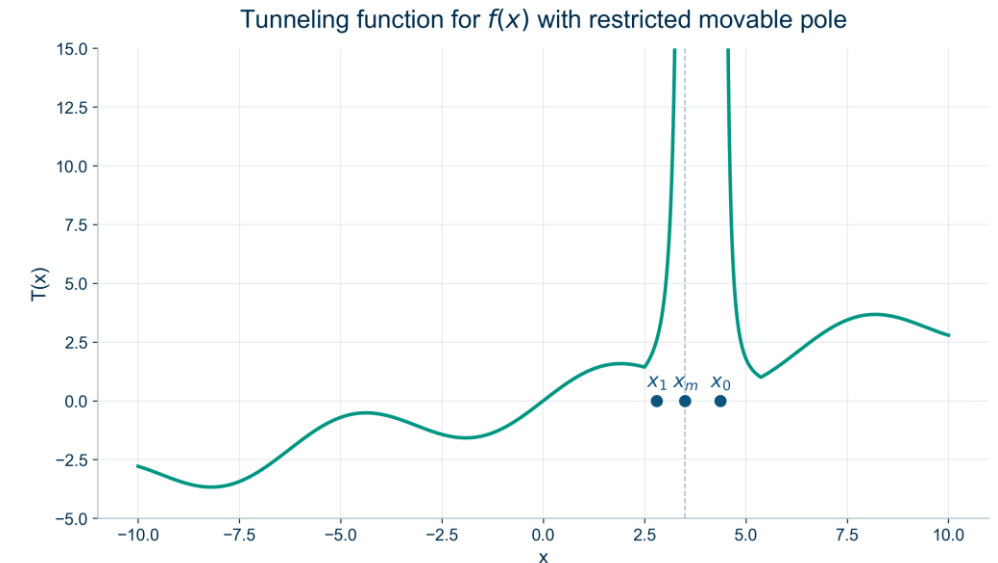
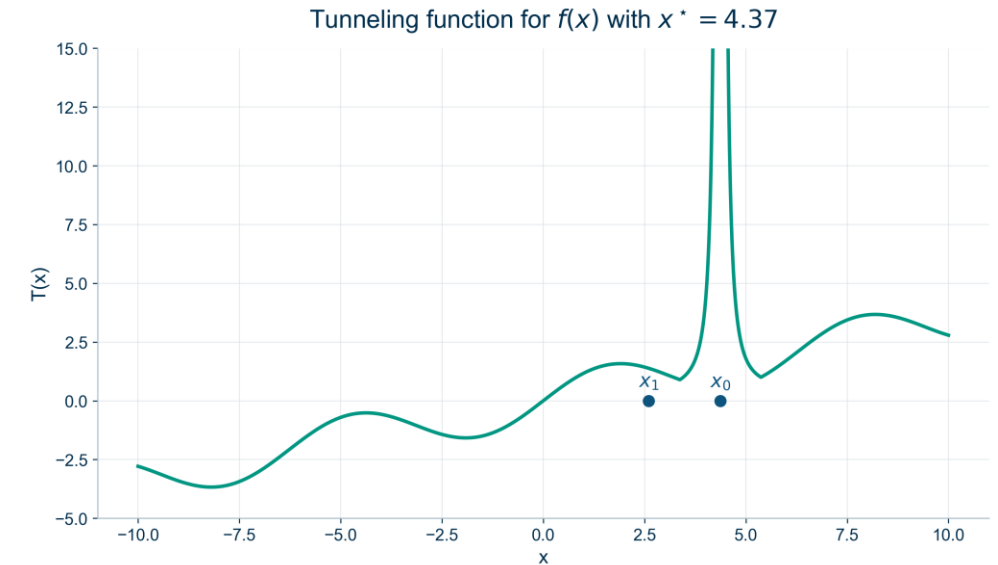
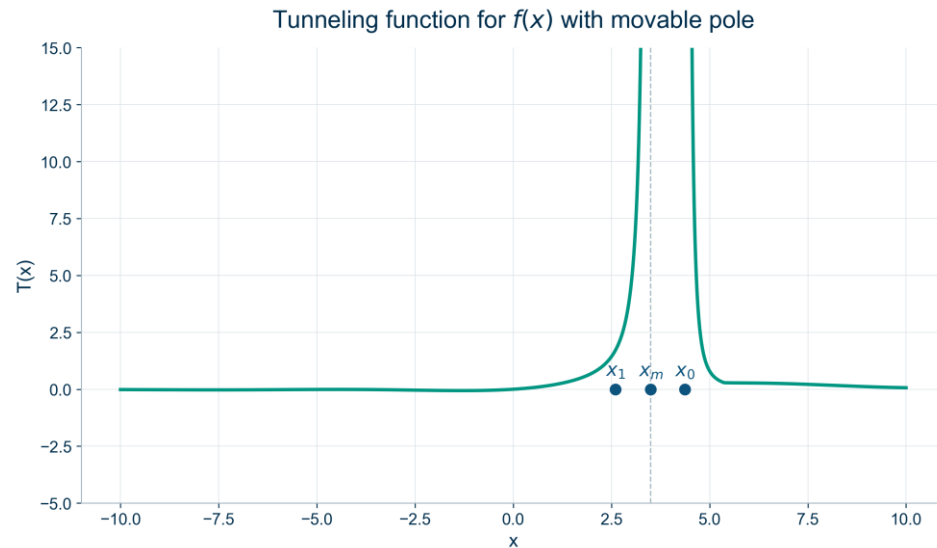
ξ_i chosen so that $\|x^{(i)} - x_m^{(i)}\| < 1$

- If no movable pole is necessary, λ_0 is reset to 0

Second Term in the Denominator

Example

- x_1 outside the radius affected by pole at $x_0 := x^*$
- Gradient at x_1 points back to the right
 - Movable pole necessary



For better comparability between functions, the impact of the movable pole is restricted to a radius of 1

How It Is Determined a Movable Pole Is Necessary

$$u_i := (x^{(i)} - x^{(i-1)})^T (\tilde{x}^{(i+1)}(\lambda_0^{(i)}) - x^{(i)}),$$

- Term u_i is evaluated
 - Scalar product of previous search direction and hypothetical next search direction
 - $u_i \geq 0$ must be satisfied
- First, checked with $\lambda_0^{(i)} = \lambda_0^{(i-1)}$
 - If $u_i \geq 0$: then checked with $\lambda_0^{(i)} = 0$
 - If $u_i < 0$: $\lambda_0^{(i)}$ iteratively increased instead until $u_i \geq 0$
- $x^{(i)}$: iterate in iteration i
- $x^{(i-1)}$: iterate in iteration $i-1$
- $\lambda_0^{(i)}$: movable pole exponent in iteration i
- $\tilde{x}^{(i+1)}(\lambda_0)$: function of what $x^{(i+1)}$ would be if determined with a movable pole with the exponent λ_0

Tunneling Phase

- Goal: Find a point x with $T(x) \leq 0$
 - As good as or better than the best point we found so far
 - Tunneling phase terminates as soon as one is found
- Starting points for search:
 - $x_i^* + \epsilon$ (previous minimal point plus small perturbation)
 - $2n$ attempts
 - Random point
 - $2n$ attempts
- Search method: restoration algorithm
 - Apply one iteration of a modified Newton's method on $T(x)$
 - Update movable pole
 - Repeat

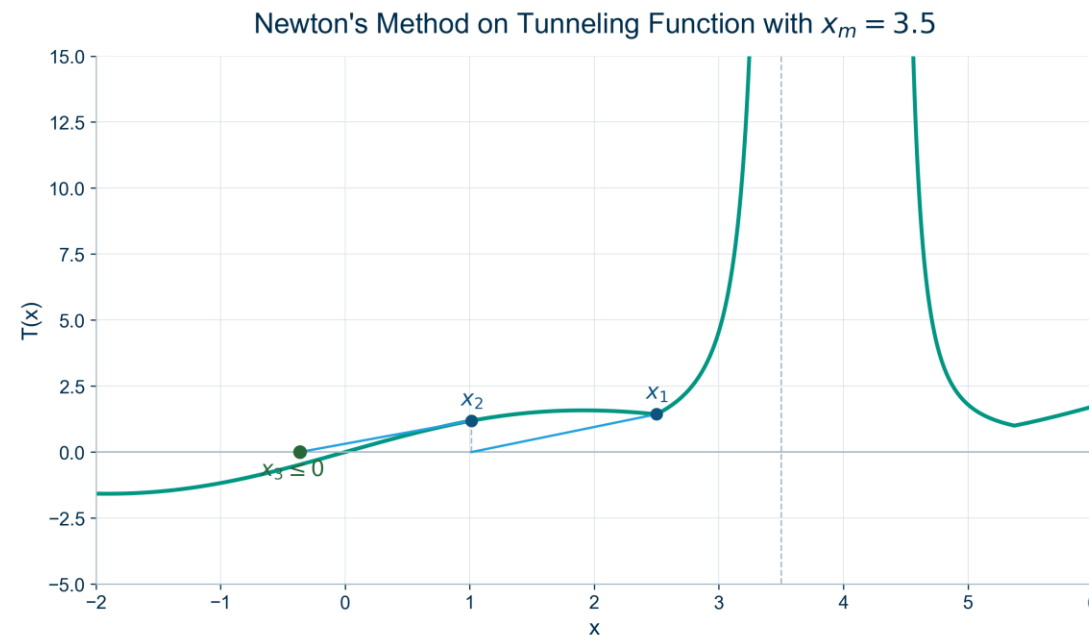
Algorithm Find New Optimal Point

```
1: for  $k = 1$  to  $2n$  do
2:    $x_0 = x_i^* + \epsilon_k$ 
3:   if RestorationAlgorithm( $x_0$ ) yields feasible  $x^*$  then return  $x^*$ 
4: for  $k = 1$  to  $2n$  do
5:    $x_0 = \text{RandomPoint}()$ 
6:   if RestorationAlgorithm( $x_0$ ) yields feasible  $x^*$  then return  $x^*$ 
7: Terminate tunneling algorithm
```

Modified Newton's Method

- Specific form of gradient descent
- Bisection parameter α , starting at 1
- Equal to regular Newton's method in one-dimensional case

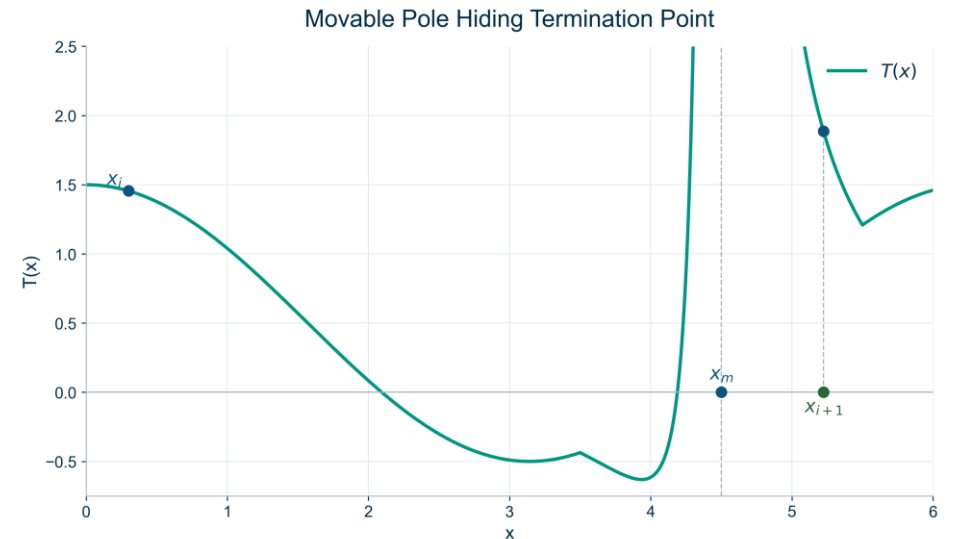
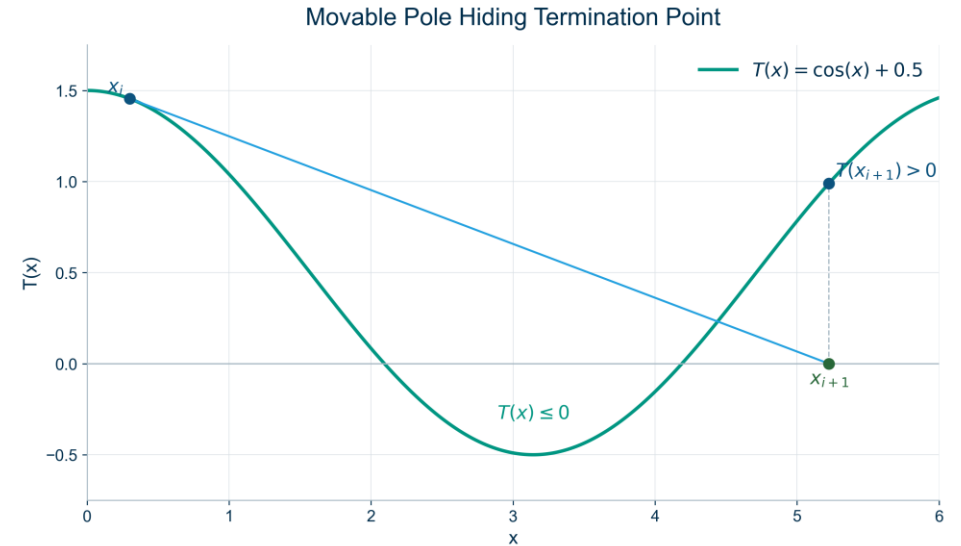
$$x_{i+1} = -\alpha \frac{T(x_i)}{\|\nabla T(x_i)\|^2} \nabla T(x_i) + x_i$$



The Movable Pole Hiding Valid Termination Points

Problems with the Algorithm

- Movable poles can hide valid termination points
 - Prevent stepping backwards
 - Multidimensional case: iterates can circle back
 - One-dimensional case: termination point will not be found
- In this example:
 - x_i not influenced by any pole
 - Movable pole added between x_i and x_{i+1} to maintain search direction
 - Valid termination point for tunneling phase skipped
- Issue of overshooting arises with nonconvex functions
 - Function lies below gradient



Stagnation Caused by the Movable Pole

Problems with the Algorithm

We examine a one-dimensional example where the iterates are far from minimal points with poles:

$$T(x) = \frac{f(x) - f^*}{(x - x_m)^{2\lambda_0}}.$$

The derivative is:

$$T'(x) = \frac{f'(x)(x - x_m)^{2\lambda_0} - (f(x) - f^*)2\lambda_0(x - x_m)^{2\lambda_0-1}}{(x - x_m)^{4\lambda_0}}$$

When $f(x) - f^* \gg f'(x)(x - x_m)$, the displacement between two consecutive iterates is:

$$\begin{aligned}\Delta x &= -\alpha \frac{T(x)}{\|T'(x)\|^2} T'(x) \\ &= -\alpha \frac{f(x) - f^*}{f'(x)(x - x_m) - 2\lambda_0(f(x) - f^*)} (x - x_m) \\ &\approx \frac{1}{2\lambda_0} \alpha (x - x_m)\end{aligned}$$

Stagnation Caused by the Movable Pole

Problems with the Algorithm

$$\Delta x \approx \frac{1}{2\lambda_0} \alpha(x - x_m) \leq \frac{1}{2}(x - x_m)$$

The distance between consecutive iterates is halved with each iteration. This can quickly cause stagnation.

Excerpt of logs from a three-dimensional algorithm run where the problem arises:

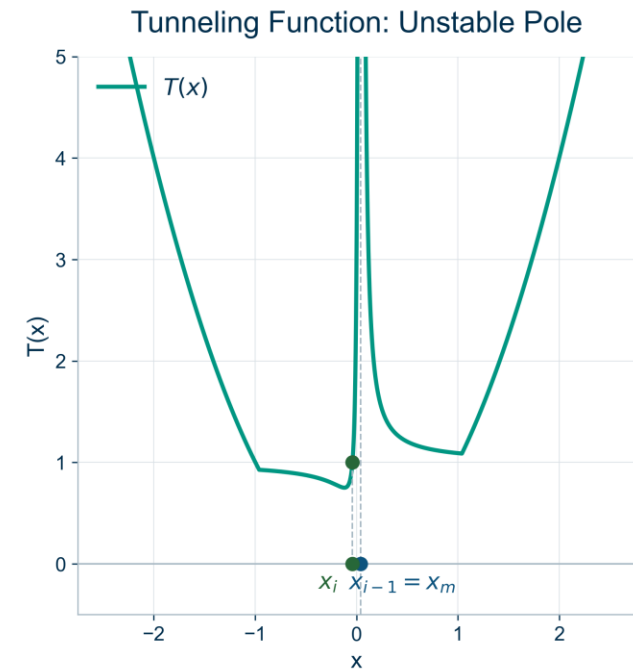
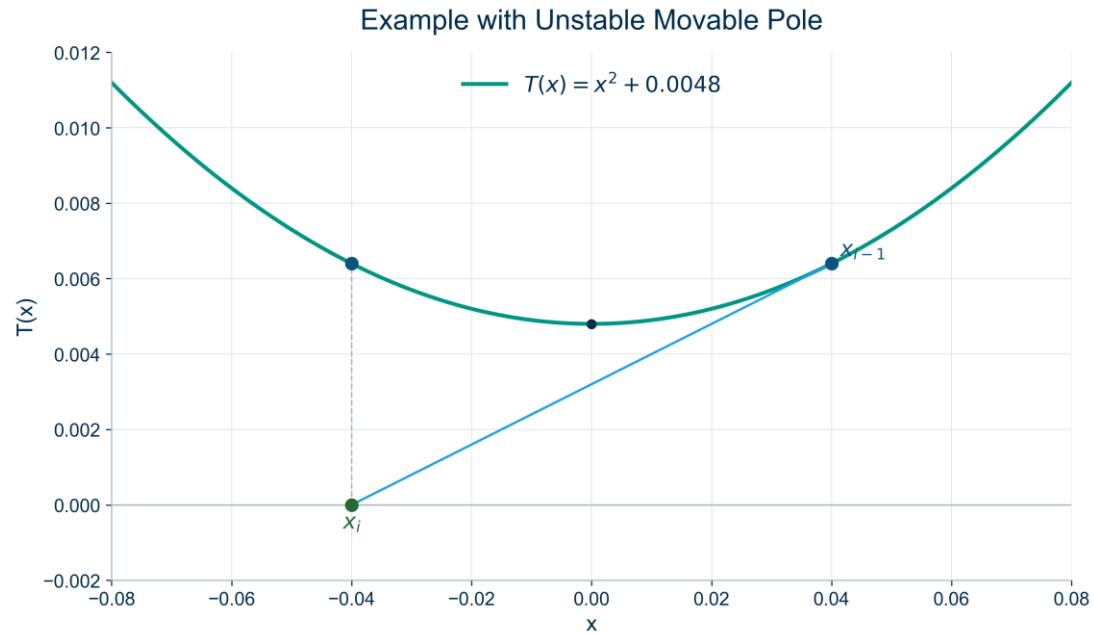
```
Next iterate of restoration algorithm: [-7.21304317 -6.45875146 -7.47856184] with T=245828.84941362668. alpha = 1.0 and delta_x = [-0.00495506 0.02873972 -0.00575734]
Next iterate of restoration algorithm: [-7.21559153 -6.44437209 -7.48151326] with T=979553.0880517605. alpha = 1.0 and delta_x = [-0.00254836 0.01437937 -0.00295141]
Next iterate of restoration algorithm: [-7.21688369 -6.43718024 -7.48300743] with T=3907752.3557766983. alpha = 1.0 and delta_x = [-0.00129216 0.00719185 -0.00149417]
Next iterate of restoration algorithm: [-7.21753429 -6.43358381 -7.48375915] with T=15607364.506701931. alpha = 1.0 and delta_x = [-0.0006506 0.00359644 -0.00075173]
Next iterate of restoration algorithm: [-7.21786073 -6.43178546 -7.48413618] with T=62379549.01275083. alpha = 1.0 and delta_x = [-0.00032643 0.00179834 -0.00037703]
Next iterate of restoration algorithm: [-7.21802423 -6.43088626 -7.48432498] with T=249415811.58438748. alpha = 1.0 and delta_x = [-0.0001635 0.0008992 -0.0001888]
Next iterate of restoration algorithm: [-7.21810605 -6.43043665 -7.48441946] with T=997455937.2772571. alpha = 1.0 and delta_x = [-8.18213091e-05 4.49608419e-04 -9.44748059e-05]
Next iterate of restoration algorithm: [-7.21814698 -6.43021185 -7.48446671] with T=3989406603.9983883. alpha = 1.0 and delta_x = [-4.09283901e-05 2.24806099e-04 -4.72556058e-05]
Next iterate of restoration algorithm: [-7.21816745 -6.43009944 -7.48449034] with T=15956789605.321524. alpha = 1.0 and delta_x = [-2.04686295e-05 1.12403521e-04 -2.36323541e-05]
Next iterate of restoration algorithm: [-7.21817768 -6.43004324 -7.48450216] with T=63825482282.91721. alpha = 1.0 and delta_x = [-1.02354234e-05 5.62018781e-05 -1.18173149e-05]
Next iterate of restoration algorithm: [-7.2181828 -6.43001514 -7.48450807] with T=255298574338.72498. alpha = 1.0 and delta_x = [-5.11798887e-06 2.81009684e-05 -5.90894193e-06]
...
```

Solution to Stagnation

- First idea: increase initial step size bisection parameter α
 - Distance between iterates still dependent on previous distances
 - Higher λ_0 or lower α decreasing one step size still affect all following iterations
 - Leads to more overshooting
- Second idea: fixing $\|x - x_m\| = 0.9$
 - x_m still set to be on the line from x_i to x_{i-1}
 - Still satisfies $\|x - x_m\| < 1$
 - Prevents previous approximation where $\|x - x_m\|$ vanishes from fraction
 - Experimentally better results

Unstable Movable Pole

Example



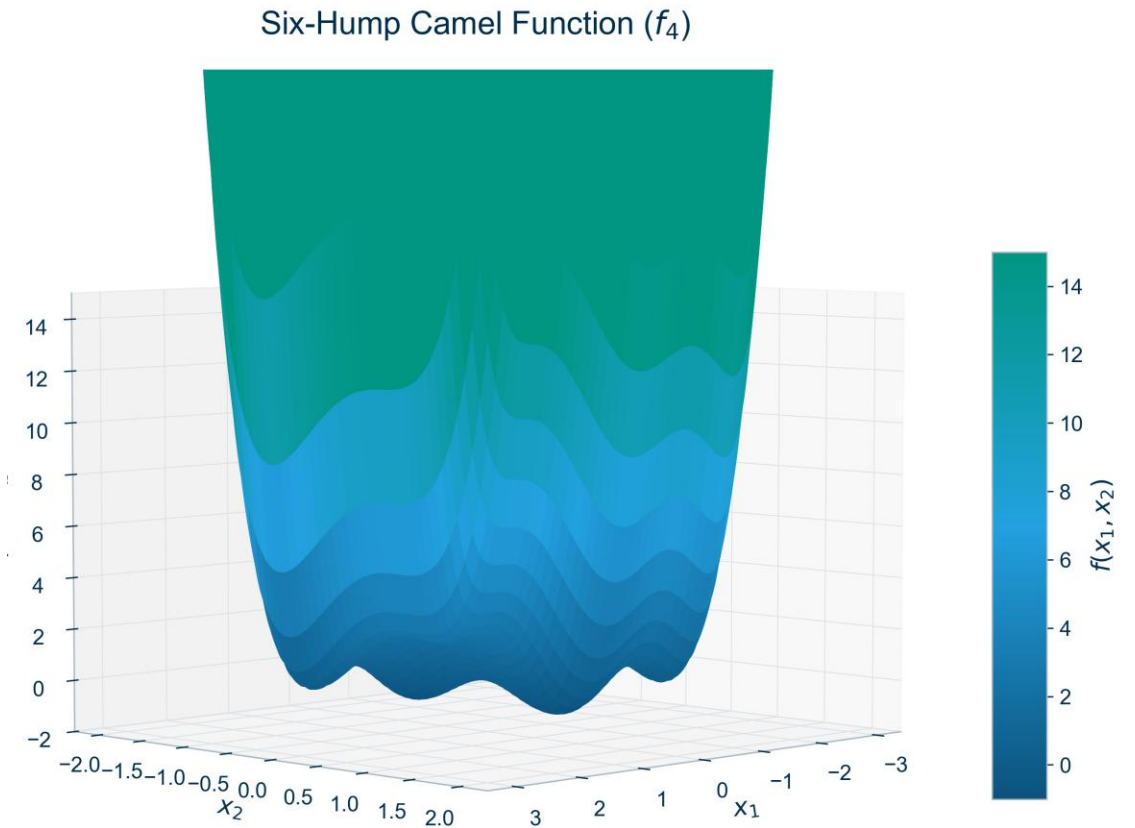
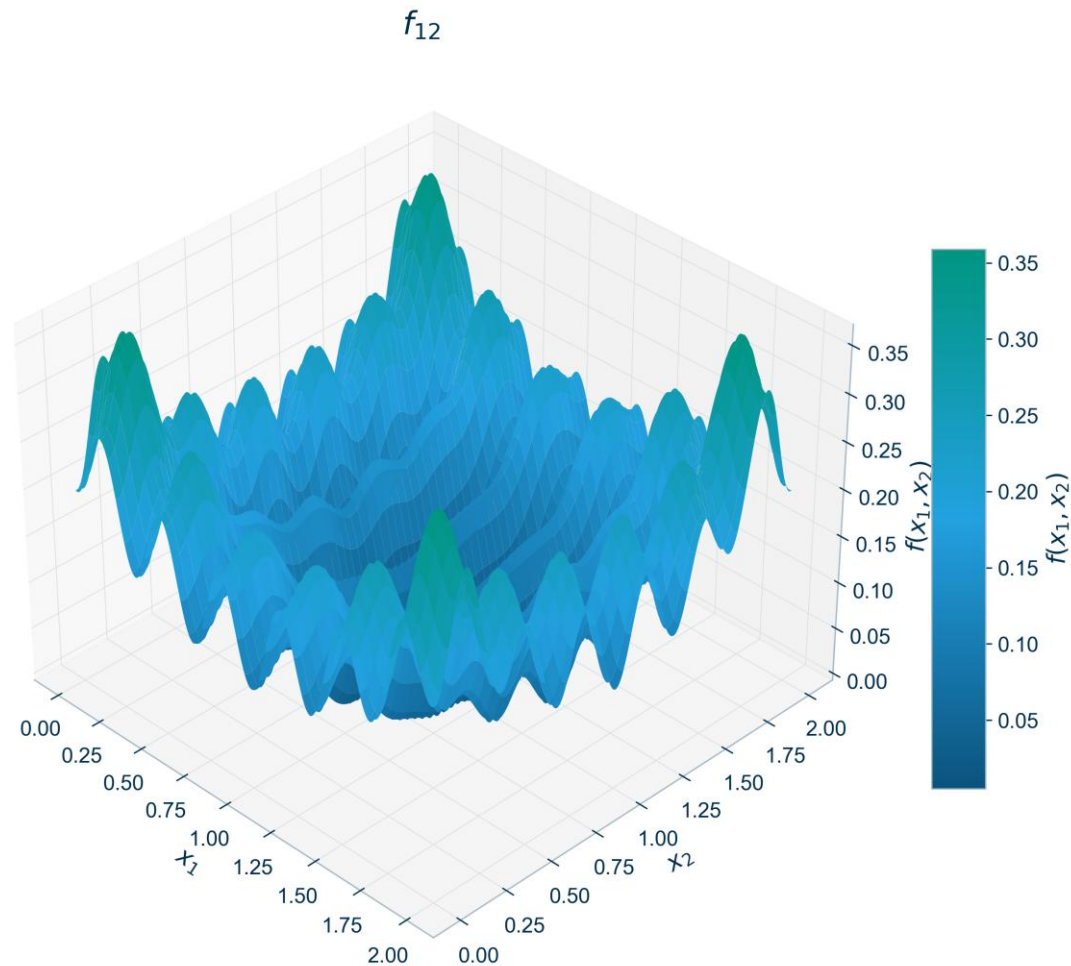
Experimental Setup

- Algorithm implementation in Python, with and without movable pole adaptation
- Tested on 17 different functions with 270 algorithm runs altogether

Function	Runs	Bounds	Minimal Value	Amt. of Minima
f_1	40	$[-10, 10]^2$	-186.731	18
f_2	10	$[-10, 10]^3$	-2709.094	81
f_3	10	$[-10, 10]^6$	-8272701.378	4374
f_4	30	$[-3, 3] \times [-2, 2]$	-1.032	2
f_5	30	$[-10, 10]^2$	-186.731	1
f_6	30	$[-10, 10]^2$	0.0	1
f_7	20	$[-10, 10]^3$	0.0	1
f_8	10	$[-10, 10]^4$	0.0	1
f_9	10	$[-10, 10]^5$	0.0	1
f_{10}	10	$[-10, 10]^8$	0.0	1
f_{11}	10	$[-10, 10]^{10}$	0.0	1
f_{12}	10	$[-10, 10]^2$	0.0	1
f_{13}	10	$[-10, 10]^3$	0.0	1
f_{14}	10	$[-10, 10]^4$	0.0	1
f_{15}	10	$[-10, 10]^5$	0.0	1
f_{16}	10	$[-10, 10]^6$	0.0	1
f_{17}	10	$[-10, 10]^7$	0.0	1

Table 1: Test functions

Example Test Functions



Comparison of Results

Test	Avg. Minima Found	Failure Rate	Time/Trial
f_1	1.52 / 18	17.50%	0.467 s
f_2	0.40 / 81	60.00%	0.458 s
f_3	0.00 / 4374	100.00%	0.711 s
f_4	1.47 / 2	0.00%	0.082 s
f_5	0.13 / 1	86.67%	0.341 s
f_6	1.00 / 1	0.00%	0.132 s
f_7	0.85 / 1	15.00%	0.233 s
f_8	0.20 / 1	80.00%	0.436 s
f_9	0.50 / 1	50.00%	0.497 s
f_{10}	0.10 / 1	90.00%	0.516 s
f_{11}	0.20 / 1	80.00%	0.870 s
f_{12}	0.40 / 1	60.00%	0.113 s
f_{13}	0.00 / 1	100.00%	0.255 s
f_{14}	0.00 / 1	100.00%	0.244 s
f_{15}	0.30 / 1	70.00%	0.350 s
f_{16}	0.20 / 1	80.00%	0.391 s
f_{17}	0.00 / 1	100.00%	0.442 s

Table 3: Performance metrics for unaltered tunneling algorithm.

Test	Avg. Minima Found	Failure Rate	Time/Trial
f_1	13.68 / 18	0.00%	1.834 s
f_2	13.30 / 81	0.00%	1.031 s
f_3	3.80 / 4374	60.00%	1.303 s
f_4	2.00 / 2	0.00%	0.206 s
f_5	0.93 / 1	6.67%	0.506 s
f_6	1.00 / 1	0.00%	0.158 s
f_7	1.00 / 1	0.00%	0.243 s
f_8	1.00 / 1	0.00%	0.328 s
f_9	1.00 / 1	0.00%	0.473 s
f_{10}	1.00 / 1	0.00%	1.052 s
f_{11}	1.00 / 1	0.00%	0.933 s
f_{12}	1.00 / 1	0.00%	0.194 s
f_{13}	1.00 / 1	0.00%	0.327 s
f_{14}	1.00 / 1	0.00%	0.416 s
f_{15}	1.00 / 1	0.00%	0.458 s
f_{16}	1.00 / 1	0.00%	0.534 s
f_{17}	1.00 / 1	0.00%	0.592 s

Table 4: Performance metrics for adapted tunneling algorithm.

Tunneling Algorithm as an Iterated Local Search Algorithm

One perspective:

- $\text{Perturbation}(s^*, \text{history})$: tunneling phase
- $\text{LocalSearch}(s')$: minimization phase
- $\text{AcceptanceCriterion}(s^*, s^{*'}, \text{history})$: always accepts

Alternatively, with lines 5 and 6 swapped:

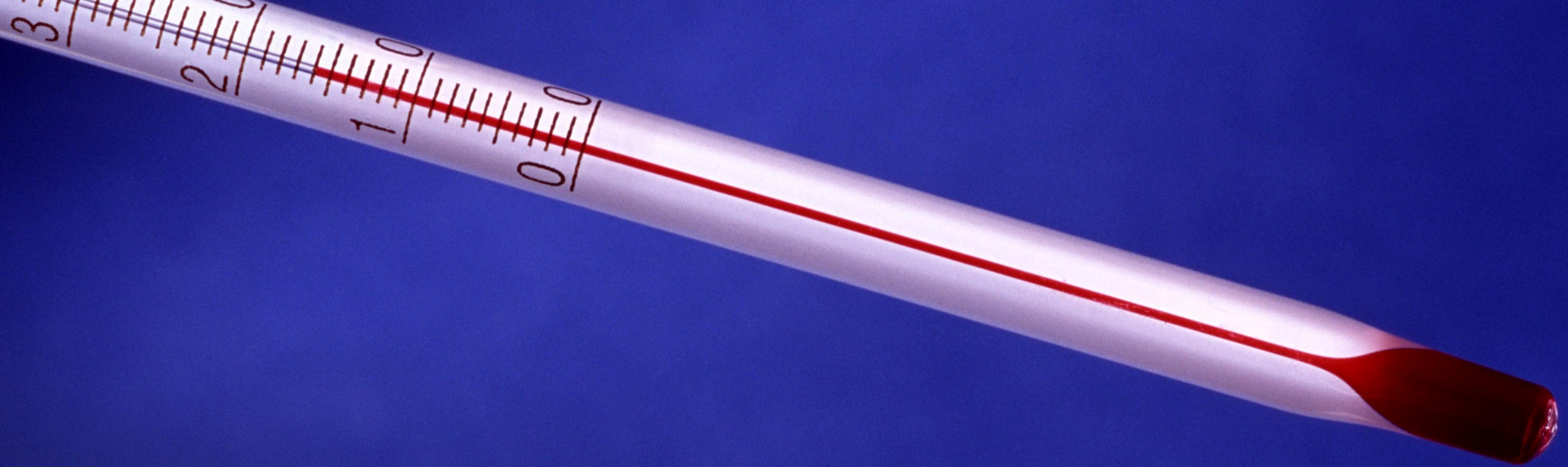
- $\text{Perturbation}(s^*, \text{history})$: one iteration of tunneling phase
- $\text{AcceptanceCriterion}(s^*, s^{*'}, \text{history})$: Is $T(x) \leq 0$?
- $\text{LocalSearch}(s')$: minimization phase

Either way:

- Termination condition: tunneling phase fails to find a new acceptable point

Algorithm Iterated local search

```
1:  $s_0 = \text{GenerateInitialSolution}$ 
2:  $s^* = \text{LocalSearch}(s_0)$ 
3: repeat
4:    $s' = \text{Perturbation}(s^*, \text{history})$ 
5:    $s^{*'} = \text{LocalSearch}(s')$ 
6:    $s^* = \text{AcceptanceCriterion}(s^*, s^{*'}, \text{history})$ 
7: until termination condition met
```



Simulated Annealing

Section
3

What It Is

- Metaheuristic for global optimization problems
 - Implementable with iterated local search
 - Core concept invented in 1953 by Metropolis et al.
 - Term coined in 1983
- Inspired by annealing from materials science
 - Slow cooling process used on metal and glass
 - Increases ductility and reduces hardness
 - Removes structural defects in atomic structure
 - Process used since 8000 BC

How It Works

- In iterated local search: changes $\text{AcceptanceCriterion}(s^*, s^{*'}, history)$ so that points s^{*}' with $f(s^{*}') > f(s^*)$ can still be accepted
 - Not possible in tunneling algorithm
- Introduces a temperature c that controls acceptance probability
 - Starts high
 - Decreases with time
 - Acceptance probability often calculated with Metropolis criterion:

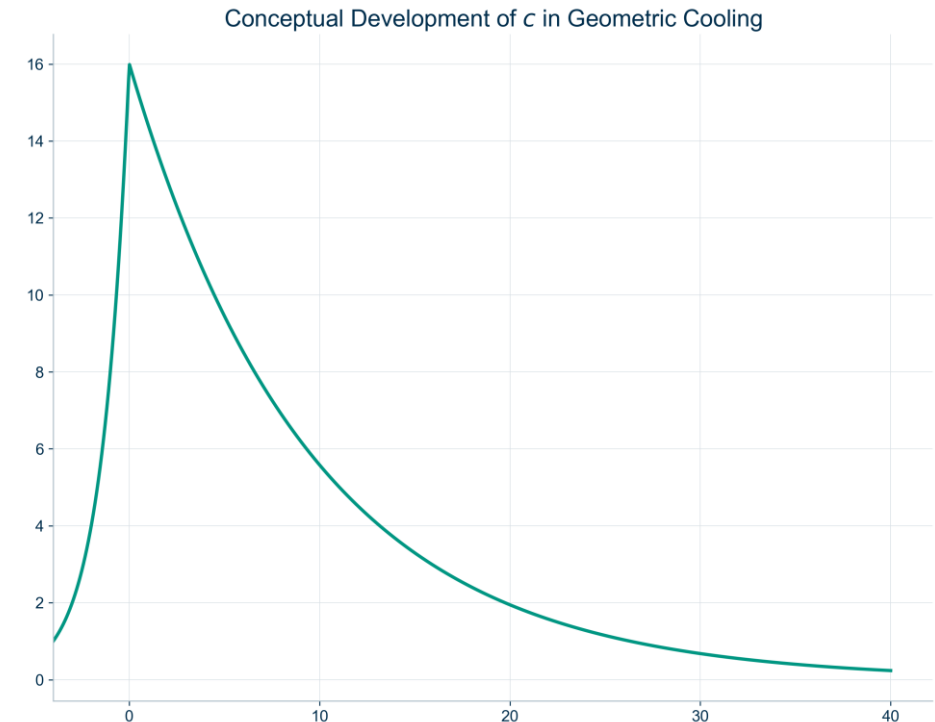
$$P\{\text{accept } x_j\} = \begin{cases} 1 & \text{if } f(x_j) < f(x_i) \\ \exp\left(\frac{f(x_i) - f(x_j)}{c}\right) & \text{else} \end{cases}$$

Geometric Cooling

Geometric cooling is a type of simulated annealing

- Initial temperature c_0 set so almost all transitions are accepted right away
 - Usually, one starts with a low temperature
 - That temperature is multiplied with a factor greater than 1 until the acceptance rate is sufficiently high
- Exponential temperature decay
 - Typically, $0.8 < \alpha < 0.99$

$$c_i := \alpha \times c_{i-1}$$



Implementation on the Tunneling Algorithm

- Initial temperature c_0 heuristically determined:
 - 100 random points sampled and objective function values calculated
 - Difference between lowest and tenth lowest objective function value taken as estimate for typical transition
 - Probability of annealing step set to 90%
- Geometric cooldown parameter α set to 0.95
 - c_i decreased with each tunneling phase
- Movable pole removed
 - Less of a risk of getting stuck in local minimal points with additional termination criterion
 - Reinstated for the first iteration of tunneling phases

$$P\{\text{accept } x_{10}\} = \exp \frac{f(x_1) - f(x_{10})}{c_0} \stackrel{!}{=} 90\%$$
$$\Rightarrow c_0 = \frac{f(x_1) - f(x_{10})}{\ln 0.9}$$

Results

Test	Avg. Minima Found	Failure Rate	Time/Trial
f_1	1.52 / 18	17.50%	0.467 s
f_2	0.40 / 81	60.00%	0.458 s
f_3	0.00 / 4374	100.00%	0.711 s
f_4	1.47 / 2	0.00%	0.082 s
f_5	0.13 / 1	86.67%	0.341 s
f_6	1.00 / 1	0.00%	0.132 s
f_7	0.85 / 1	15.00%	0.233 s
f_8	0.20 / 1	80.00%	0.436 s
f_9	0.50 / 1	50.00%	0.497 s
f_{10}	0.10 / 1	90.00%	0.516 s
f_{11}	0.20 / 1	80.00%	0.870 s
f_{12}	0.40 / 1	60.00%	0.113 s
f_{13}	0.00 / 1	100.00%	0.255 s
f_{14}	0.00 / 1	100.00%	0.244 s
f_{15}	0.30 / 1	70.00%	0.350 s
f_{16}	0.20 / 1	80.00%	0.391 s
f_{17}	0.00 / 1	100.00%	0.442 s

Table 3: Performance metrics for unaltered tunneling algorithm.

Test	Avg. Minima Found	Failure Rate	Time/Trial
f_1	10.85 / 18	0.00%	0.877 s
f_2	5.10 / 81	0.00%	1.105 s
f_3	0.10 / 4374	90.00%	1.267 s
f_4	1.50 / 2	0.00%	0.118 s
f_5	0.67 / 1	33.33%	0.388 s
f_6	1.00 / 1	0.00%	0.421 s
f_7	0.95 / 1	5.00%	1.698 s
f_8	0.90 / 1	10.00%	2.303 s
f_9	1.00 / 1	0.00%	2.141 s
f_{10}	0.80 / 1	20.00%	3.584 s
f_{11}	0.80 / 1	20.00%	6.316 s
f_{12}	1.00 / 1	0.00%	0.512 s
f_{13}	1.00 / 1	0.00%	1.147 s
f_{14}	1.00 / 1	0.00%	2.114 s
f_{15}	1.00 / 1	0.00%	3.144 s
f_{16}	1.00 / 1	0.00%	3.048 s
f_{17}	1.00 / 1	0.00%	4.653 s

Table 5: Performance metrics for tunneling algorithm with simulated annealing.

What Is Happening in the Background

- Regular tunneling phase: descent steps with modified Newton's method on complex tunneling function
- With annealing:
 - Tunneling phases are terminated prematurely
 - Switch to more efficient minimization algorithms
 - Simplification of function geometry
 - Stagnation interrupted
- New termination criterion for tunneling phase
 - Less of a risk of tunneling algorithm terminating when tunneling phase fails to find a termination point
- Risk of wasting time rediscovering already known minimal points

Conclusion

Results

What Worked Well

- Pinning the movable pole to certain distance from iterates
 - Numerical stability
- Simulated annealing for problems with many minima
 - Algorithm less likely to terminate too early
- Replacing movable pole with other anti-stagnation measures

What Did Not Work So Well

- Simulated annealing for problems with few minima
 - Algorithm less likely to terminate when it should
- Experimental results worse than those reported by Levy and Montalvo
 - Cause unknown

Experimental Results Comparison

Test	Avg. Minima Found
f_1	17 / 18
f_2	n/a
f_3	n/a
f_4	2 / 2
f_5	1 / 1
f_6	1 / 1
f_7	1 / 1
f_8	1 / 1
f_9	1 / 1
f_{10}	1 / 1
f_{11}	1 / 1
f_{12}	0.5 / 1
f_{13}	0.75 / 1
f_{14}	0.75 / 1
f_{15}	0.75 / 1
f_{16}	1 / 1
f_{17}	0.75 / 1

Table 2: Performance metrics reported by Levy and Montalvo.

Test	Avg. Minima Found	Failure Rate	Time/Trial
f_1	13.68 / 18	0.00%	1.834 s
f_2	13.30 / 81	0.00%	1.031 s
f_3	3.80 / 4374	60.00%	1.303 s
f_4	2.00 / 2	0.00%	0.206 s
f_5	0.93 / 1	6.67%	0.506 s
f_6	1.00 / 1	0.00%	0.158 s
f_7	1.00 / 1	0.00%	0.243 s
f_8	1.00 / 1	0.00%	0.328 s
f_9	1.00 / 1	0.00%	0.473 s
f_{10}	1.00 / 1	0.00%	1.052 s
f_{11}	1.00 / 1	0.00%	0.933 s
f_{12}	1.00 / 1	0.00%	0.194 s
f_{13}	1.00 / 1	0.00%	0.327 s
f_{14}	1.00 / 1	0.00%	0.416 s
f_{15}	1.00 / 1	0.00%	0.458 s
f_{16}	1.00 / 1	0.00%	0.534 s
f_{17}	1.00 / 1	0.00%	0.592 s

Table 4: Performance metrics for adapted tunneling algorithm.

Test	Avg. Minima Found	Failure Rate	Time/Trial
f_1	1.52 / 18	17.50%	0.467 s
f_2	0.40 / 81	60.00%	0.458 s
f_3	0.00 / 4374	100.00%	0.711 s
f_4	1.47 / 2	0.00%	0.082 s
f_5	0.13 / 1	86.67%	0.341 s
f_6	1.00 / 1	0.00%	0.132 s
f_7	0.85 / 1	15.00%	0.233 s
f_8	0.20 / 1	80.00%	0.436 s
f_9	0.50 / 1	50.00%	0.497 s
f_{10}	0.10 / 1	90.00%	0.516 s
f_{11}	0.20 / 1	80.00%	0.870 s
f_{12}	0.40 / 1	60.00%	0.113 s
f_{13}	0.00 / 1	100.00%	0.255 s
f_{14}	0.00 / 1	100.00%	0.244 s
f_{15}	0.30 / 1	70.00%	0.350 s
f_{16}	0.20 / 1	80.00%	0.391 s
f_{17}	0.00 / 1	100.00%	0.442 s

Table 3: Performance metrics for unaltered tunneling algorithm.

Test	Avg. Minima Found	Failure Rate	Time/Trial
f_1	10.85 / 18	0.00%	0.877 s
f_2	5.10 / 81	0.00%	1.105 s
f_3	0.10 / 4374	90.00%	1.267 s
f_4	1.50 / 2	0.00%	0.118 s
f_5	0.67 / 1	33.33%	0.388 s
f_6	1.00 / 1	0.00%	0.421 s
f_7	0.95 / 1	5.00%	1.698 s
f_8	0.90 / 1	10.00%	2.303 s
f_9	1.00 / 1	0.00%	2.141 s
f_{10}	0.80 / 1	20.00%	3.584 s
f_{11}	0.80 / 1	20.00%	6.316 s
f_{12}	1.00 / 1	0.00%	0.512 s
f_{13}	1.00 / 1	0.00%	1.147 s
f_{14}	1.00 / 1	0.00%	2.114 s
f_{15}	1.00 / 1	0.00%	3.144 s
f_{16}	1.00 / 1	0.00%	3.048 s
f_{17}	1.00 / 1	0.00%	4.653 s

Table 5: Performance metrics for tunneling algorithm with simulated annealing.

